



Fire Detection - Alert System

DEEP_LEARNING

Professeur :
Anass BELCAID

Préparer par :
Anas ELKHATIB
Ayoub ANHAL



10 janvier 2025

Table des matières

Remerciements	3
Dédicace	4
Introduction	5
1 Applications	6
1.1 Domaines d'application	6
2 Dataset	8
2.1 Présentation des données	8
2.2 FIRE	9
2.3 NON-FIRE	10
2.4 base de données	13
3 Bibliothèques Utilisées	15
3.1 Gestion des Données	15
3.2 Visualisation	16
3.3 Machine Learning et Deep Learning	16
3.4 Évaluation des Performances	16
4 Pré-Traitement des Données	17
4.1 Préparation des Données	17
4.1.1 process_dataset	17
4.1.2 walk_through_dir	18
4.2 Traitement données et création du DataFrame final	19
4.2.1 Initialisation des chemins des datasets	19
4.2.2 Création du DataFrame final	19
4.2.3 Affichage des informations du DataFrame final	20
4.3 Mappage des labels et mise à jour du DataFrame	20
4.3.1 Mappage des labels	20
4.3.2 Remplacement des labels dans le DataFrame	20
4.3.3 Résultat du DataFrame final	21
4.4 Interprétation des Résultats du DataFrame :	21
4.4.1 Structure du DataFrame	21
4.4.2 Interprétation des Données	22
4.5 Augmentation des Données	22
4.5.1 Devition des Donnees	22

4.5.2	Augmentation	23
4.5.3	Généralisation	23
4.6	Préparation Avant l'Entraînement du Modèle	25
4.6.1	Fonctions d'évaluation des performances	25
4.6.2	Redimensionner et Normaliser du Modèle	27
4.6.3	Stratégie d'arrêt précoce du Modèle	27
4.6.4	Fonction entraînement du Modèle	28
5	Algorithmes	30
5.1	Modifications sur Model CNN	30
5.2	Comprendre input et output Model CNN	33
5.2.1	1_ère couche convolution	33
5.2.2	2_ème couche convolution	34
5.2.3	3_ème couche convolution	35
5.2.4	4_ème couche convolution	36
5.2.5	Flatten et couche de classification	37
5.2.6	Output de model	38
5.3	entraînement du Model	38
5.3.1	Les paramètres et entraînement du Model	38
5.3.2	Resultat d'entraînement du Model	39
5.4	Test du Modèle	40
5.4.1	Prediction	40
5.5	évaluation du Modèle	42
5.5.1	Préparation de l'Image	42
5.5.2	Prediction	43
5.5.3	Exemples	44
6	Comparaison	46
6.1	Description des blocs de l'architecture AlexNet	46
6.2	Description des blocs de l'architecture VGGNet	48
6.3	Tableau de Comparaison	51
7	Conclusion	52

Remerciements

Avant tout développement sur cette expérience professionnelle, il apparaît opportun de commencer ce rapport par des remerciements à ceux qui nous ont beaucoup appris au cours de ce rapport.

Celui qui ne remercie pas les gens ne remercie pas Dieu, donc au terme de ce travail, on tient à remercier Pr. ANASS BELCAID notre professeur qui 'a proposé ce projet et nous a conseillés sur l'orientation que celui-ci devait prendre avec beaucoup de patience et de pédagogie. Bref, on remercie toute personne qui nous a prêté aide ou assistance de près ou de loin.

Dédicace

On dédie ce modeste travail à :

En premier lieu à ceux que personne ne peut compenser les sacrifices qu'ils ont consentis pour notre éducation et notre bien-être à nos parents qui se sont sacrifiés pour nous prendre en charge tout au long de notre formation et qui sont à l'origine de notre réussite que Dieu les Garde et les protège.

À notre famille et nos chers amis qui nous ont accordé leur soutien dans les instants les plus difficiles. Tous nos formateurs et toute l'équipe pédagogique.

Introduction

Définition du Problème

La détection d'incendie est essentielle tant pour les applications personnelles que commerciales. Les systèmes traditionnels basés sur des capteurs de chaleur, de pression, de fumée et de température sont lents et peuvent représenter un danger en cas de retard. En revanche, les systèmes de détection d'incendie vidéo, utilisant la vision par ordinateur et l'intelligence artificielle, offrent une alternative plus rapide et précise. Ces systèmes peuvent détecter le feu en temps réel, localiser sa source et envoyer des alertes instantanées, contribuant ainsi à une réponse plus rapide et efficace en cas d'urgence.

Introduction de Projet

De manière générale, ce projet vise à utiliser des algorithmes d'apprentissage automatique de pointe pour servir les communautés. Le temps de réponse des unités d'urgence est probablement le facteur numéro un qui détermine si quelqu'un s'en sortira vivant ou non. Ce projet utilise un système de surveillance autonome en temps réel qui détecte le feu et la fumée à l'aide de l'intelligence artificielle et de l'apprentissage automatique. Le but ultime de ce projet serait de réduire le taux de mortalité routière en détectant les anomalies, principalement les incendies et les fumées, et en les signalant aux autorités correspondantes.

1. Applications

1.1 Domaines d'application

Selon la National Fire Protection Association (NFPA), en moyenne, il y a eu un civil tué par le feu toutes les 2 heures et 24 minutes et un civil blessé par le feu toutes les 35 minutes. Les incendies représentaient 4 pourcent du total des appels d'urgence. C'est pourquoi notre projet de détection de feu et d'envoi d'alerte peut être appliqué de nombreuses façons.



FIGURE 1.1 – SYSTÈMES DE SÉCURITÉ RÉSIDENTIELS

Les systèmes de sécurité résidentiels équipés de détection de feu permettent d'alerter rapidement les occupants en cas d'incendie, réduisant les risques de pertes humaines et matérielles. Ils peuvent inclure des alarmes connectées et des notifications en temps réel.



FIGURE 1.2 – SURVEILLANCE INDUSTRIELLE

surveillance industrielle détectent rapidement les incendies dans les zones à haut risque, minimisant les interruptions et les dégâts. Ils intègrent des capteurs avancés et des alertes en temps réel pour une intervention rapide.



FIGURE 1.3 – SYSTÈMES DE TRANSPORT ET INFRASTRUCTURES

Transport et infrastructures utilisent des solutions de détection de feu pour prévenir les incendies dans les tunnels, gares ou véhicules. Ils assurent la sécurité des usagers grâce à des alertes rapides et une intervention ciblée.

2. Dataset

2.1 Présentation des données

La détection d'incendies et de fumée présente plusieurs défis en raison de la disponibilité limitée de données. Les ensembles de données vidéo existants ne sont pas entièrement exploitables car ils se composent principalement de séries d'images extraites de vidéos. Pour qu'un modèle de détection fonctionne efficacement, il doit apprendre non seulement à identifier les événements d'incendie et de fumée, mais aussi à différencier ces événements d'autres phénomènes naturels ou artificiels

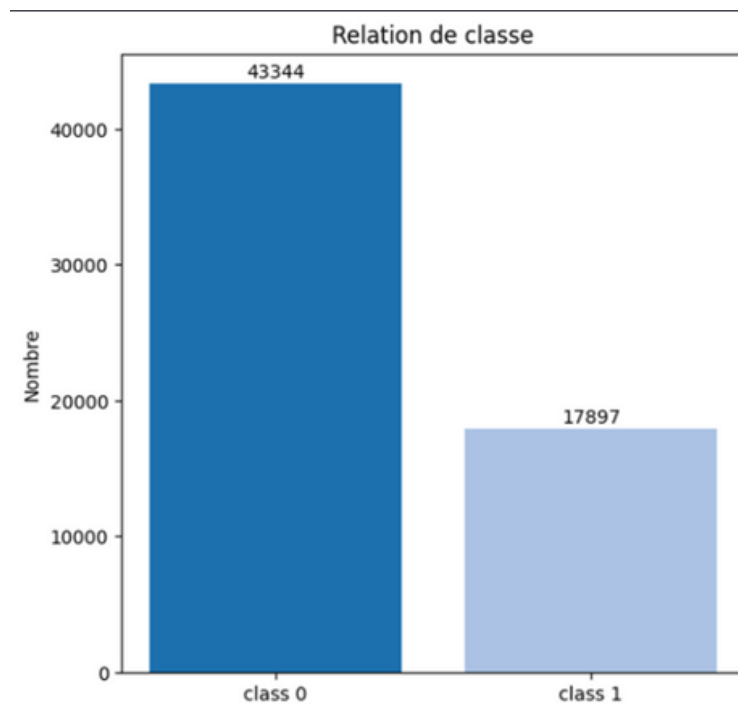


FIGURE 2.1 – Figure 4 : Répartition des classes dans le jeu de données

Classe "class 0" : Elle est associée à la catégorie nonfire (pas de feu), représentant 43,344 instances dans le jeu de données.

Classe "class 1" : Elle est associée à la catégorie fire (feu), représentant 17,897 instances.

2.2 FIRE

Le réseau est censé détecter les incendies et la fumée dans les zones industrielles, les zones routières et les calamités naturelles. L'ensemble de données doit contenir suffisamment de détails sur ces événements et représenter une bonne quantité d'éléments de feu et de fumée dans la scène pour être visibles sur la caméra.

type d'image collecte :



FIGURE 2.2 – BÂTIMENTS RÉSIDENTIELS

L'image montre des bâtiments résidentiels vulnérables aux incendies, en particulier ceux qui pourraient être exposés à des risques d'incendie dans des zones urbaines ou suburbaines.



FIGURE 2.3 – ZONES INDUSTRIELLES

L'image illustre des zones industrielles où des incendies peuvent se déclencher en raison de l'équipement lourd, des matières inflammables et des processus de fabrication.



FIGURE 2.4 – FORÊT

L'image montre une forêt susceptible de subir des feux de forêt, avec des conditions propices à la propagation du feu à travers la végétation dense et sèche.



FIGURE 2.5 – SYSTÈMES DE TRANSPORT

L'image représente des infrastructures de transport potentiellement exposées aux risques d'incendie, en particulier celles impliquant des véhicules motorisés ou des installations industrielles.

2.3 NON-FIRE

Nous avons collecté plusieurs images représentant des couleurs, des maisons, des rues, ainsi que des scènes d'éclairage en journée et de nuit, afin d'éviter que le modèle ne commette des erreurs, comme prédire la couleur rouge ou une lampe comme étant un incendie.

Types d'images collectées :

Éclairage

Cette image montre des sources de lumière artificielle, telles que des lampes ou des éclairages de rue, qui pourraient être confondues avec des flammes en raison de leur intensité lumineuse.



FIGURE 2.6 – Éclairage

Rues (Streets)



FIGURE 2.7 – Rues urbaines (Streets)

L'image représente une rue urbaine éclairée par des lampadaires ou des enseignes lumineuses. Ces halos lumineux peuvent être confondus avec des flammes par le système de détection.

Couleurs

- **Rouge** : Zones ou objets rouges qui, en raison de leur couleur vive, pourraient être confondus avec du feu, sans présenter de danger réel. - **Orange** : Objets ou surfaces de couleur orange ressemblant à des flammes, mais non liés à un incendie. - **Jaune** : Éléments lumineux jaunes pouvant tromper le système de détection. - **Blanc** : Objets blancs ou reflets lumineux pouvant être interprétés à tort comme des éclats de flammes.

Maisons (Houses)

Les images de cette catégorie montrent des situations où des éléments visuels, comme des lumières, des reflets ou des décorations, peuvent être mal interprétés comme des incendies.

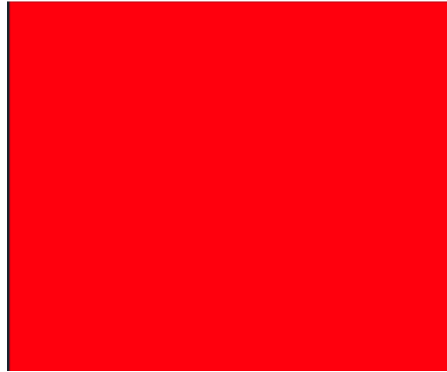


FIGURE 2.8 – Couleurs



FIGURE 2.9 – Maisons (Houses)

Nuit (Night Time)



FIGURE 2.10 – Nuit (Night Time)

Les images nocturnes mettent en évidence des scénarios où des sources lumineuses artificielles, telles que des lampadaires, des phares de véhicules ou des enseignes lumineuses, ainsi que des reflets sur des surfaces brillantes, peuvent être confondus avec des flammes.

Matin (Morning Time)



FIGURE 2.11 – Matin (Morning Time)

Les images matinales illustrent des situations où les conditions lumineuses, comme les reflets du soleil sur des fenêtres, des voitures ou des bâtiments, produisent des éclats brillants semblables à des flammes, pouvant induire le système en erreur.

2.4 base de données

Les données ont été extraites de la plateforme Kaggle :

- Feu : Images de scènes impliquant des incendies.
- Normal : Images de scènes sans incendie.

Collecte des données : 16 datasets couvrant différents contextes (forêt, rue, maisons, entreprises).

Équilibre : Images avec et sans feu pour une bonne généralisation du modèle.

Éclairage : Données variées incluant des scènes de jour, de nuit et différents niveaux de luminosité.

Gestion des couleurs : Datasets spécifiques pour éviter la confusion avec des teintes similaires au feu.

SPLIT (TRAIN/TEST) :

- TRAIN SPLIT : 80%
- TEST SPLIT : 20%

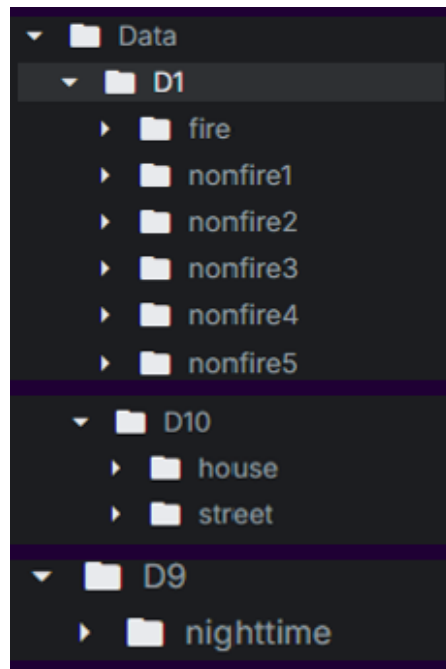


FIGURE 2.12 – Dataset

Dataset Path : <https://www.kaggle.com/datasets/anhelayoub/fire-data>

3. Bibliothèques Utilisées

Listing 3.1 – Bibliothèques Python utilisées dans le projet

```
1 # gestion des donn es
2 import os
3 import sys
4 from pathlib import Path
5 import numpy as np
6 import pandas as pd
7 from PIL import Image
8
9 # visualisation
10 import seaborn as sns
11 import matplotlib.pyplot as plt
12 import itertools
13 from collections import Counter
14
15 # Machine Learning et Deep Learning
16 import tensorflow as tf
17 from tensorflow import keras
18 from tensorflow.keras import layers, models
19 from tensorflow.keras.models import Model
20 from tensorflow.keras.optimizers import Adam
21 from tensorflow.keras.callbacks import Callback, EarlyStopping, ModelCheckpoint
22 from tensorflow.keras.layers import Rescaling, Normalization, CenterCrop, RandomFlip
23 from tensorflow.keras.preprocessing.image import ImageDataGenerator
24 from keras.layers import Dense, Dropout, Conv2D, MaxPooling2D, Flatten,
    BatchNormalization
25
26 # l' valuation des performances
27 from sklearn.model_selection import train_test_split
28 from sklearn.metrics import classification_report, confusion_matrix
```

3.1 Gestion des Données

os : Fournit une manière d’interagir avec le système d’exploitation, permettant de manipuler des fichiers et des répertoires.

sys : Permet d’accéder à des variables et fonctions spécifiques à l’interpréteur Python.

pathlib : Fournit des classes pour manipuler les chemins de fichiers de manière orientée objet.

numpy : Bibliothèque pour le calcul numérique, offrant des structures de données puissantes comme les tableaux multidimensionnels.

pandas : Utilisée pour la manipulation et l’analyse de données, offrant des structures de données comme les DataFrames.

PIL(Pillow) : Bibliothèque pour le traitement d'images, permettant d'ouvrir, de manipuler et de sauvegarder des images.

3.2 Visualisation

seaborn : Bibliothèque de visualisation de données basée sur Matplotlib, facilitant la création de graphiques informatifs.

matplotlib : Bibliothèque de traçage 2D pour créer des visualisations statiques, animées et interactives.

itertools : Fournit des fonctions pour créer des itérateurs efficaces pour boucles.

collections : Fournit des alternatives aux types de données intégrés, comme Counter pour compter les occurrences.

3.3 Machine Learning et Deep Learning

tensorflow : Bibliothèque open-source pour le calcul numérique et l'apprentissage automatique.

keras : API de haut niveau pour construire et entraîner des modèles de deep learning.

layers : Module de Keras contenant des couches de réseau de neurones.

models : Module de Keras pour définir des modèles de réseaux de neurones.

optimizers : Fournit des algorithmes pour optimiser les modèles d'apprentissage.

callbacks : Permet d'ajouter des fonctionnalités supplémentaires lors de l'entraînement des modèles.

ImageDataGenerator : Générateur de données pour l'augmentation d'images.

3.4 Évaluation des Performances

sklearn.model_selection : Fournit des outils pour diviser les données en ensembles d'entraînement et de test. Permet d'effectuer une validation croisée pour évaluer la robustesse du modèle.

sklearn.metrics : Fournit des fonctions pour évaluer les performances des modèles, telles que :

- Le rapport de classification, qui inclut des métriques comme la précision, le rappel et le score F1.
- La matrice de confusion, qui permet de visualiser les performances du modèle en termes de vrais positifs, faux positifs, vrais négatifs et faux négatifs.

4. Pré-Traitement des Données

4.1 Préparation des Données

```
1 def process_dataset(dataset_path):
2     image_dir = Path(dataset_path)
3     filepaths = list(image_dir.glob('**/*.JPG')) + list(image_dir.glob('**/*.jpg')) +
4                 list(image_dir.glob('**/*.png'))
5     labels = list(map(lambda x: os.path.split(os.path.split(x)[0])[1], filepaths))
6     filepaths = pd.Series(filepaths, name='Filepath').astype(str)
7     labels = pd.Series(labels, name='Label')
8     dataset_df = pd.concat([filepaths, labels], axis=1)
9
10    return dataset_df
11
12 def walk_through_dir(dataset_paths):
13     all_datasets = pd.DataFrame(columns=['Filepath', 'Label'])
14
15     for dataset_path in dataset_paths:
16         print("-" * 90)
17         print(f"Processing directory: {dataset_path}")
18         dataset_df = process_dataset(dataset_path)
19         all_datasets = pd.concat([all_datasets, dataset_df], ignore_index=True)
20
21    return all_datasets
```

4.1.1 process_dataset

La fonction `process_dataset` prend en entrée un chemin d'accès à un répertoire de données d'images (`dataset_path`) et effectue les étapes suivantes pour préparer les données en vue de leur utilisation dans un projet de détection de feu.

Étapes de la fonction

1. **Récupération des fichiers d'images** : La fonction utilise `pathlib` pour rechercher tous les fichiers d'images (`.JPG`, `.jpg`, `.png`) dans le répertoire et ses sous-répertoires. Cela est effectué en utilisant la méthode `glob` avec un modèle de recherche récursive (`**/*`) pour trouver tous les fichiers avec les extensions spécifiées.

```
1     filepaths = list(image_dir.glob('**/*.JPG')) + list(image_dir.glob('**/*.jpg'))
2                 + list(image_dir.glob('**/*.png'))
```

2. **Extraction des labels** : Le label de chaque image est extrait en utilisant le nom du répertoire parent du fichier d'image. Cela est effectué en utilisant `os.path.split` pour obtenir le répertoire parent, puis en extrayant son nom qui est utilisé comme label.

```
1 labels = list(map(lambda x: os.path.split(os.path.split(x)[0])[1], filepaths))
```

3. **Création d'un DataFrame** : Un DataFrame est créé à partir des chemins des fichiers d'images (`filepaths`) et des labels associés (`labels`). Ces deux listes sont combinées dans un DataFrame avec `pandas`.

```
1 filepaths = pd.Series(filepaths, name='Filepath').astype(str)
2 labels = pd.Series(labels, name='Label')
3 dataset_df = pd.concat([filepaths, labels], axis=1)
```

Retour de la fonction

La fonction retourne un DataFrame combiné contenant deux colonnes :

- **Filepath** : Chemin d'accès complet de chaque image dans tous les répertoires spécifiés.
- **Label** : Le label associé à chaque image (par exemple, `fire`, `non fire`, `smoke`).

Cela permet de centraliser et d'organiser les données d'images provenant de différents répertoires dans un seul jeu de données.

4.1.2 walk_through_dir

La fonction `walk_through_dir` prend en entrée une liste de chemins vers plusieurs répertoires de données d'images (`dataset_paths`) et parcourt chaque répertoire pour collecter les informations sur les images et leurs labels. Elle utilise la fonction `process_dataset` pour chaque répertoire et combine les résultats dans un seul DataFrame.

Étapes de la fonction

1. **Initialisation du DataFrame** : Un DataFrame vide nommé `all_datasets` est créé avec les colonnes `Filepath` et `Label`, pour accumuler les données de tous les répertoires traités.

```
1 all_datasets = pd.DataFrame(columns=['Filepath', 'Label'])
```

2. **Traitement de chaque répertoire** : La fonction parcourt la liste des répertoires spécifiés dans `dataset_paths`. Pour chaque répertoire, elle appelle la fonction `process_dataset` pour extraire les informations des fichiers d'images et de leurs labels. Le DataFrame résultant est ensuite concaténé à `all_datasets` pour accumuler les données des différents répertoires.

```
1 for dataset_path in dataset_paths:
2     print("-" * 90)
3     print(f"Processing directory: {dataset_path}")
4     dataset_df = process_dataset(dataset_path)
5     all_datasets = pd.concat([all_datasets, dataset_df], ignore_index=True)
```

3. **Retour de la fonction** : Après avoir traité tous les répertoires, la fonction retourne un DataFrame combiné contenant les chemins d'accès à toutes les images et leurs labels associés. Cela permet d'avoir une vue d'ensemble de toutes les données de tous les répertoires spécifiés dans `dataset_paths`.

```
1 return all_datasets
```

Retour de la fonction

La fonction retourne un `DataFrame` contenant deux colonnes :

- **Filepath** : Chemin d'accès des images.
- **Label** : Le label associé à chaque image (par exemple, `fire`, `non fire`, `smoke`).

Cela permet d'organiser les données d'images de manière structurée pour une utilisation ultérieure dans des modèles de détection de feu.

4.2 Traitement données et création du `DataFrame` final

Cette partie du code est utilisée pour traiter un ensemble de répertoires contenant des données d'images (représentant des catégories telles que `fire`, `non fire`, et `smoke`) et combiner ces données en un seul `DataFrame` à l'aide de la fonction `walk_through_dir`.

4.2.1 Initialisation des chemins des datasets

Une liste de chemins de répertoires `dataset_paths` est définie, chacun pointant vers un sous-répertoire contenant un ensemble d'images. Ces répertoires sont situés sur la plateforme Kaggle, et chaque répertoire est désigné par un nom tel que `D1`, `D2`, ..., `D16`.

```
1 dataset_paths = [  
2     "/kaggle/input/fire-data/Data/D1",  
3     "/kaggle/input/fire-data/Data/D2",  
4     "/kaggle/input/fire-data/Data/D3",  
5     "/kaggle/input/fire-data/Data/D4",  
6     "/kaggle/input/fire-data/Data/D5",  
7     "/kaggle/input/fire-data/Data/D6",  
8     "/kaggle/input/fire-data/Data/D7",  
9     "/kaggle/input/fire-data/Data/D8",  
10    "/kaggle/input/fire-data/Data/D9",  
11    "/kaggle/input/fire-data/Data/D10",  
12    "/kaggle/input/fire-data/Data/D11",  
13    "/kaggle/input/fire-data/Data/D12",  
14    "/kaggle/input/fire-data/Data/D13",  
15    "/kaggle/input/fire-data/Data/D14",  
16    "/kaggle/input/fire-data/Data/D15",  
17    "/kaggle/input/fire-data/Data/D16"  
18 ]
```

4.2.2 Création du `DataFrame` final

La fonction `walk_through_dir` est ensuite appelée avec la liste `dataset_paths` comme argument. Cette fonction parcourt tous les répertoires listés et extrait les chemins des images ainsi que leurs labels en utilisant la fonction `process_dataset`, comme expliqué précédemment. Les résultats de chaque répertoire sont concaténés dans un `DataFrame` final.

```
1 final_dataset = walk_through_dir(dataset_paths)
```


4.2.3 Affichage des informations du DataFrame final

Enfin, la méthode `info()` est utilisée sur le `DataFrame` final pour afficher un résumé des données, y compris le nombre d'entrées, les types de données et le nombre de valeurs non nulles dans chaque colonne.

```
1 print(final_dataset.info())
```

Cela permet de vérifier que les données ont été correctement combinées et préparées pour les étapes suivantes du projet, comme l'entraînement du modèle de détection de feu.

4.3 Mappage des labels et mise à jour du DataFrame

Cette partie du code est utilisée pour remplacer les labels catégoriels par des valeurs numériques afin de préparer les données pour l'entraînement d'un modèle de machine learning. L'objectif est de convertir les labels sous forme de chaînes de caractères (par exemple, `'fire'`, `'nonfire1'`) en valeurs numériques (0 ou 1) qui peuvent être utilisées par les algorithmes de machine learning.

4.3.1 Mappage des labels

Un dictionnaire de mappage, `label_mapping`, est défini pour associer chaque label de catégorie à une valeur numérique. Dans ce cas, tous les labels associés à des images de type `'nonfire'`, `'house'`, `'street'`, etc., sont mappés à la valeur 0, tandis que le label `'fire'` est mappé à la valeur 1.

```
1 label_mapping = {
2     'nonfire': 0,
3     'nonfire1': 0,
4     'nonfire2': 0,
5     'nonfire3': 0,
6     'nonfire4': 0,
7     'nonfire5': 0,
8     'nofire': 0,
9     'house': 0,
10    'street': 0,
11    'nighttime': 0,
12    'ntime': 0,
13    'mtime': 0,
14    'red': 0,
15    'green': 0,
16    'test': 0,
17    'D16': 0,
18    'fire': 1
19 }
```

4.3.2 Remplacement des labels dans le DataFrame

La méthode `replace()` de `pandas` est utilisée pour remplacer les labels dans la colonne `Label` du `DataFrame` `final_dataset` selon le dictionnaire de mappage. Chaque valeur de label est remplacée par sa valeur numérique correspondante définie dans `label_mapping`.

```
1 final_dataset["Label"] = final_dataset["Label"].replace(label_mapping)
```

Cela permet de convertir les labels textuels en labels numériques, ce qui est essentiel pour les modèles de machine learning qui nécessitent des entrées sous forme numérique. Après cette étape, la colonne `Label` du `DataFrame` `final_dataset` contiendra des valeurs 0 (pour 'nonfire' et autres labels similaires) et 1 (pour 'fire').

4.3.3 Résultat du DataFrame final

Le `DataFrame` `final_dataset` contient maintenant les chemins des images dans la colonne `Filepath` et les labels numériques correspondants dans la colonne `Label`. Ce `DataFrame` est prêt à être utilisé pour l'entraînement et la validation du modèle de détection de feu.

4.4 Interprétation des Résultats du DataFrame :

	Filepath	Label
0	/kaggle/input/fire-data/Data/D1/nonfire4/nonfi...	0
1	/kaggle/input/fire-data/Data/D1/nonfire4/nonfi...	0
2	/kaggle/input/fire-data/Data/D1/nonfire4/nonfi...	0
3	/kaggle/input/fire-data/Data/D1/nonfire2/nonfi...	0
4	/kaggle/input/fire-data/Data/D1/nonfire2/nonfi...	0
...	...	...
61236	/kaggle/input/fire-data/Data/D16/nonfire80.png	0
61237	/kaggle/input/fire-data/Data/D16/nonfire121.png	0
61238	/kaggle/input/fire-data/Data/D16/nonfire115.png	0
61239	/kaggle/input/fire-data/Data/D16/nonfire164.png	0
61240	/kaggle/input/fire-data/Data/D16/nonfire129.png	0
61241 rows × 2 columns		

FIGURE 4.1 – Format de Dataset

Le résultat du `DataFrame` obtenu après le traitement des données est structuré en deux colonnes principales : `Filepath` et `Label`. Voici une analyse détaillée du contenu de ce `DataFrame` :

4.4.1 Structure du DataFrame

Le `DataFrame` final contient **61241 lignes** et **2 colonnes** :

- **Filepath** : Cette colonne contient les chemins d'accès des images dans le répertoire de données. Chaque entrée représente l'emplacement d'une image sur le système de fichiers, par exemple `/kaggle/input/fire-data/Data/D1/nonfire4/nonfire1.png`.
- **Label** : Cette colonne contient les labels numériques associés à chaque image. Les labels sont convertis en valeurs numériques avec le dictionnaire de mappage, où 0 correspond à une image de catégorie `nonfire` et 1 correspond à une image de catégorie `fire`.

4.4.2 Interprétation des Données

1. Type des Images :

- Les images dont le label est 0 (par exemple, `nonfire4`, `nonfire2`, etc.) appartiennent à la catégorie `nonfire`, indiquant qu'il ne s'agit pas d'images de feu.
- Aucune image n'a de label 1 dans le `DataFrame` actuel, ce qui signifie qu'aucune image de feu (`fire`) n'est incluse dans ce sous-ensemble de données.

2. Données des Images :

- La colonne `Filepath` montre l'emplacement exact des images sur le disque. Par exemple, `/kaggle/input/fire-data/Data/D1/nonfire4/nonfire1.png` indique le chemin où l'image se trouve.
- Les images sont organisées dans différents répertoires, et chaque répertoire correspond à une catégorie particulière de label (`nonfire4`, `nonfire2`, etc.).

3. Impact sur l'Entraînement du Modèle :

- Le modèle d'apprentissage automatique devra apprendre à classer les images ayant le label 0, c'est-à-dire celles de la catégorie `nonfire`.
- Bien que la majorité des images dans le jeu de données soient de type `nonfire`, ce qui pourrait sembler déséquilibré, cela peut en fait être bénéfique pour le modèle. En effet, les images de la catégorie `nonfire` couvrent une grande diversité de scènes, telles que les conditions de nuit, différentes couleurs d'éclairage, des scènes de rue, et d'autres environnements variés. Cette diversité permet au modèle d'apprendre des caractéristiques plus générales et robustes pour cette classe.
- Les images de la catégorie `fire` (label 1) sont généralement plus simples à détecter, car elles représentent un phénomène spécifique et bien connu. En revanche, les données `nonfire` peuvent varier considérablement, ce qui rend l'apprentissage de cette catégorie plus complexe et intéressant pour le modèle. Il est donc crucial que le modèle soit exposé à une grande variété d'images `nonfire` pour bien distinguer les scènes de feu des autres types de scènes.

4.5 Augmentation des Données

4.5.1 Devition des Donnees

```
1 train_df, test_df = train_test_split(final_dataset, test_size=0.15, shuffle=True,
    random_state=42)
```

Cette ligne divise le `final_dataset` en deux ensembles : un ensemble d'entraînement (`train_df`) et un ensemble de test (`test_df`).

- `test_size=0.15` : signifie que 15% des données seront utilisées pour l'ensemble de test, tandis que 85% des données seront utilisées pour l'entraînement.
- `shuffle=True` : signifie que les données seront mélangées avant la division.
- `random_state=42` : fixe la graine pour assurer la reproductibilité du processus de division.

4.5.2 Augmentation

```
1 train_generator = ImageDataGenerator(  
2     rescale=1./255,  
3     rotation_range=20,  
4     width_shift_range=0.2,  
5     height_shift_range=0.2,  
6     shear_range=0.2,  
7     zoom_range=0.2,  
8     horizontal_flip=True,  
9     fill_mode='nearest',  
10    validation_split=0.2  
11 )
```

Crée un générateur d'images pour l'ensemble d'entraînement (**train_df**) avec plusieurs options de transformation pour augmenter la diversité des images pendant l'entraînement.

- **rescale=1./255** : normalise les valeurs des pixels des images pour les ramener entre 0 et 1.
- **rotation_range=20** : permet une rotation aléatoire des images jusqu'à 20 degrés.
- **width_shift_range=0.2** et **height_shift_range=0.2** déplacent aléatoirement les images horizontalement et verticalement.
- **shear_range=0.2** : applique une transformation de cisaillement pour déformer les images.
- **zoom_range=0.2** : applique un zoom aléatoire sur les images.
- **horizontal_flip=True** : effectue un retournement horizontal aléatoire des images.
- **fill_mode='nearest'** : définit la méthode de remplissage pour les pixels vides créés par certaines transformations.
- **validation_split=0.2** : réserve 20% des données d'entraînement pour la validation.

```
1 test_generator = ImageDataGenerator(  
2     rescale=1./255  
3 )
```

Crée un générateur d'images pour l'ensemble de test (**test_df**), avec une normalisation des valeurs des pixels entre 0 et 1. Aucune transformation supplémentaire n'est appliquée aux images de test.

4.5.3 Généralisation

```
1 train_images = train_generator.flow_from_dataframe(  
2     dataframe=train_df,  
3     x_col='Filepath',  
4     y_col='Label',  
5     target_size=(224, 224),  
6     color_mode='rgb',  
7     class_mode='raw',  
8     batch_size=32,  
9     shuffle=True,  
10    seed=42,  
11    subset='training',  
12 )  
13  
14 val_images = train_generator.flow_from_dataframe(  
15     dataframe=train_df,  
16     x_col='Filepath',  
17     y_col='Label',
```

```

18     target_size=(224, 224),
19     color_mode='rgb',
20     class_mode='raw',
21     batch_size=32,
22     shuffle=True,
23     seed=42,
24     subset='validation'
25 )
26
27 test_images = test_generator.flow_from_dataframe(
28     dataframe=test_df,
29     x_col='Filepath',
30     y_col='Label',
31     target_size=(224, 224),
32     color_mode='rgb',
33     class_mode='raw',
34     batch_size=32,
35     shuffle=False
36 )

```

Ce code permet de créer des générateurs d'images pour l'entraînement, la validation et les tests. La fonction **flow_from_dataframe** de Keras est utilisée dans tous les cas pour charger les images et leurs étiquettes à partir d'un **DataFrame**. La principale différence entre chaque générateur réside dans le dataset utilisé (par exemple, **train_df**, **test_df**) et le subset choisi (entraînement, validation ou test).

- **dataframe=data_df** : Ce paramètre spécifie le **DataFrame** contenant les chemins d'accès des images et les labels associés. Selon l'utilisation, ce **DataFrame** peut être **train_df**, **val_df** ou **test_df**.
- **x_col='Filepath'** : La colonne **Filepath** contient les chemins d'accès aux images.
- **y_col='Label'** : La colonne **Label** contient les étiquettes associées aux images (par exemple, 0 pour nonfeu, 1 pour feu).
- **target_size=(224, 224)** : Ce paramètre redimensionne les images à la taille de 224x224 pixels avant de les transmettre au modèle.
- **color_mode='rgb'** : Les images sont chargées en mode couleur RGB (Red, Green, Blue).
- **class_mode='raw'** : Le mode **raw** signifie que les labels sont traités comme des valeurs numériques, adaptées aux problèmes de régression ou de classification binaire.
- **batch_size=32** : Le générateur crée des lots de 32 images à chaque itération pour les utiliser dans le modèle.
- **shuffle=True** : Les images sont mélangées avant chaque époque pour améliorer la généralisation du modèle en évitant les biais d'ordre.
- **seed=42** : Ce paramètre fixe la graine aléatoire pour garantir que les résultats soient reproductibles à chaque exécution.
- **subset='subset_name'** : Ce paramètre définit si le générateur est destiné à l'entraînement, à la validation ou aux tests. Cela permet de spécifier quel sous-ensemble de données sera utilisé.

4.6 Préparation Avant l'Entraînement du Modèle

4.6.1 Fonctions d'évaluation des performances

Fonction accuracy

```
1 def accuracy(y_true, y_pred):
2     correct_predictions = tf.reduce_sum(tf.cast(tf.equal(y_true, tf.round(y_pred)), tf.
        float32))
3     total_samples = tf.cast(tf.size(y_true), tf.float32)
4     accuracy_value = correct_predictions / total_samples * 100
5     return accuracy_value
```

La fonction `accuracy` évalue la précision d'un modèle en calculant la proportion de prédictions correctes parmi l'ensemble des échantillons.

- `tf.equal(y_true, tf.round(y_pred))` : Compare les valeurs réelles (`y_true`) et les prédictions arrondies (`y_pred`). Cela génère un tableau booléen indiquant si chaque prédiction est correcte (`True`) ou incorrecte (`False`).
- `tf.reduce_sum` : Calcule la somme des valeurs `True` (correctes), qui correspond au nombre total de prédictions correctes.
- `tf.size(y_true)` : Donne le nombre total d'échantillons dans `y_true`.
- `correct_predictions / total_samples * 100` : Divise le nombre de prédictions correctes par le nombre total d'échantillons pour obtenir la précision en pourcentage.

Cette fonction est utilisée pour évaluer les performances d'un modèle en fournissant une mesure quantitative (en pourcentage) de la précision de ses prédictions sur un ensemble de données donné.

Fonction precision

```
1 def precision(y_true, y_pred, positive_class=1):
2     true_positive = np.sum((np.array(y_true) == positive_class) & (np.array(y_pred) ==
        positive_class))
3     predicted_positive = np.sum(np.array(y_pred) == positive_class)
4     precision_value = true_positive / predicted_positive if predicted_positive != 0 else
        0
5     return precision_value
```

La fonction `precision` calcule la précision, c'est-à-dire la proportion des prédictions positives correctes parmi toutes les prédictions positives effectuées par le modèle.

- `np.array(y_true)` : Convertit `y_true` en tableau NumPy pour effectuer des opérations vectorisées.
- `true_positive` : Calcule le nombre de vrais positifs (les cas où la prédiction et la vérité réelle sont toutes deux égales à `positive_class`).
- `predicted_positive` : Compte le nombre total de prédictions positives effectuées par le modèle (`y_pred == positive_class`).
- `precision_value` : Divise `true_positive` par `predicted_positive` pour obtenir la précision. Si `predicted_positive` est égal à 0 (aucune prédiction positive), la fonction renvoie 0 pour éviter une division par zéro.

La précision est utilisée pour évaluer la capacité du modèle à éviter les fausses alertes (faux positifs). Elle est particulièrement utile lorsque le coût des erreurs de fausses détections est élevé, comme dans la détection de feux.

Fonction recall

```
1 def recall(y_true, y_pred, positive_class=1):
2     true_positive = np.sum((np.array(y_true) == positive_class) & (np.array(y_pred) ==
3         positive_class))
4     actual_positive = np.sum(np.array(y_true) == positive_class)
5     recall_value = true_positive / actual_positive if actual_positive != 0 else 0
6     return recall_value
```

La fonction `recall` calcule le rappel, c'est-à-dire la proportion des échantillons positifs correctement identifiés parmi tous les échantillons réellement positifs.

- `np.array(y_true)` : Convertit `y_true` en tableau NumPy pour effectuer des opérations vectorisées.
- `true_positive` : Calcule le nombre de vrais positifs (les cas où la prédiction et la vérité réelle sont toutes deux égales à `positive_class`).
- `actual_positive` : Compte le nombre total de cas réellement positifs dans les données (`y_true == positive_class`).
- `recall_value` : Divise `true_positive` par `actual_positive` pour obtenir le rappel. Si `actual_positive` est égal à 0 (aucun cas positif dans les données), la fonction renvoie 0 pour éviter une division par zéro.

Le rappel est utilisé pour évaluer la capacité du modèle à détecter correctement les cas positifs (vrais positifs). Cela est particulièrement crucial dans des contextes où manquer des cas positifs (faux négatifs) peut avoir des conséquences graves, comme dans la détection de feux.

Fonction f1_score

```
1 def f1_score(y_true, y_pred, positive_class=1):
2     precision_value = precision(y_true, y_pred, positive_class)
3     recall_value = recall(y_true, y_pred, positive_class)
4     f1_score_value = 2 * (precision_value * recall_value) / (precision_value +
5         recall_value) if (precision_value + recall_value) != 0 else 0
6     return f1_score_value
```

La fonction `f1_score` calcule le score F1, une métrique qui combine la précision (`precision`) et le rappel (`recall`) pour évaluer les performances globales d'un modèle, particulièrement dans les cas de déséquilibre entre les classes.

- `precision_value` : Appelle la fonction `precision` pour obtenir la précision du modèle sur les données.
- `recall_value` : Appelle la fonction `recall` pour obtenir le rappel du modèle sur les données.
- `f1_score_value` : Combine la précision et le rappel en utilisant la formule :

$$F1 = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Si la somme de la précision et du rappel est égale à 0, la fonction renvoie 0 pour éviter une division par zéro.

Le score F1 est une mesure harmonique qui équilibre la précision et le rappel. Il est particulièrement utile lorsqu'un modèle doit bien détecter les cas positifs tout en limitant les faux positifs et les faux négatifs. C'est une métrique idéale pour des tâches où les données sont déséquilibrées, comme la détection de feux.

4.6.2 Redimensionner et Normaliser du Modèle

```
1 resize_and_rescale = tf.keras.Sequential([
2     tf.keras.layers.Rescaling(1./255),
3     tf.keras.layers.RandomFlip("horizontal")
4 ])
```

Le code `resize_and_rescale` définit une séquence d'opérations de prétraitement des images pour les préparer avant leur passage dans le modèle. Ces transformations permettent de normaliser les données et d'améliorer la robustesse du modèle grâce à l'augmentation des données.

- `tf.keras.layers.Rescaling(1./255)` : Cette couche redimensionne les valeurs des pixels de l'image pour les normaliser dans l'intervalle $[0, 1]$. Cela est essentiel pour que le modèle fonctionne correctement, car les réseaux neuronaux sont sensibles aux échelles des données.
- `tf.keras.layers.RandomFlip("horizontal")` : Cette couche applique un retournement horizontal aléatoire aux images. Cela constitue une méthode d'augmentation des données, augmentant la diversité des échantillons d'entraînement et réduisant le risque de surapprentissage.

Ce pipeline de prétraitement améliore la qualité des données en normalisant les pixels et en introduisant une variabilité artificielle via des transformations aléatoires. Cela aide le modèle à généraliser davantage et à être moins sensible aux variations dans les données d'entrée.

4.6.3 Stratégie d'arrêt précoce du Modèle

```
1 early_stopping = EarlyStopping(
2     monitor="val_loss",
3     patience=10,
4     verbose=1,
5     restore_best_weights=True
6 )
```

Le code `early_stopping` définit une stratégie d'arrêt précoce pour le processus d'entraînement du modèle. Cette méthode permet d'interrompre l'entraînement lorsque les performances du modèle sur les données de validation cessent de s'améliorer, afin de prévenir le surapprentissage et de réduire le temps d'entraînement.

- `monitor="val_loss"` : Cette option indique que la métrique surveillée est la perte sur le jeu de validation (`val_loss`). Lorsque cette perte cesse de diminuer, cela signifie que le modèle a atteint son point optimal.
- `patience=10` : Le paramètre `patience` spécifie le nombre d'époques à attendre avant d'arrêter l'entraînement si aucune amélioration n'est constatée. Ici, il attend 10 époques.
- `verbose=1` : Cette option contrôle le niveau de détails affichés. Avec 1, des messages sont affichés lorsqu'un arrêt précoce est déclenché.
- `restore_best_weights=True` : Cette option garantit que les poids du modèle sont restaurés à ceux de l'époque où la performance sur les données de validation était la meilleure.

La stratégie d'arrêt précoce est une technique importante pour optimiser l'entraînement du modèle. Elle empêche le modèle de s'entraîner inutilement longtemps, ce qui peut entraîner un surapprentissage, et permet de conserver les meilleurs paramètres trouvés pendant l'entraînement.

4.6.4 Fonction entraînement du Modèle

```
1 def train_model(model, train_images, val_images, epochs=20):
2     all_training_loss = []
3     all_val_loss = []
4     all_training_acc = []
5     all_val_acc = []
6
7     for epoch in range(1, epochs + 1):
8         print(f"Epoch_{epoch}/{epochs}")
9         try:
10            # Entraînement pour une époque
11            history = model.fit(
12                train_images,
13                steps_per_epoch=len(train_images),
14                validation_data=val_images,
15                validation_steps=len(val_images),
16                epochs=1,
17                callbacks=[
18                    early_stopping,
19                    checkpoint_callback
20                ]
21            )
22
23            all_training_loss.append(history.history['loss'][0])
24            all_val_loss.append(history.history['val_loss'][0])
25            all_training_acc.append(history.history['accuracy'][0])
26            all_val_acc.append(history.history['val_accuracy'][0])
27
28        except StopIteration:
29            continue
30
31        except Exception as e:
32            print(f"Une erreur s'est produite pendant l'époque {epoch}: {e}")
33            break
34
35        if early_stopping.stopped_epoch > 0:
36            print("L'entraînement a été interrompu en raison de l'arrêt précoce.")
37            break
38
39    print("L'entraînement s'est terminé.")
40
41    return {
42        "training_loss": all_training_loss,
43        "val_loss": all_val_loss,
44        "training_accuracy": all_training_acc,
45        "val_accuracy": all_val_acc
46    }
```

La fonction `train_model` encapsule le processus d'entraînement du modèle sur les données. Elle entraîne un modèle pour un nombre donné d'époques, surveille ses performances à chaque époque, et enregistre les métriques de perte (`loss`) et d'exactitude (`accuracy`) pour les données d'entraînement et de validation.

- **model.fit** : Cette méthode effectue une époque d'entraînement du modèle sur les données fournies (`train_images`) et évalue sa performance sur les données de validation (`val_images`).

- **Callbacks :**
 - `early_stopping` : Arrête l'entraînement si la perte sur les données de validation ne s'améliore plus.
 - `checkpoint_callback` : Sauvegarde le meilleur modèle à chaque époque.
- **`all_training_loss`, `all_val_loss`, `all_training_acc`, `all_val_acc` :** Ces listes enregistrent respectivement les valeurs de la perte et de l'exactitude pour les données d'entraînement et de validation à chaque époque.
- **Gestion des exceptions :**
 - `StopIteration` : Ignore les interruptions dues à un manque de données.
 - `Exception` : Capture les erreurs imprévues et arrête le processus d'entraînement en cas de problème.
- **Arrêt précoce :** Si `early_stopping` détecte une stagnation dans les performances du modèle, l'entraînement s'interrompt avant d'atteindre le nombre maximal d'époques.

Cette fonction est essentielle pour l'entraînement supervisé. Elle optimise le processus en collectant des informations sur les performances et en appliquant des techniques comme l'arrêt précoce et la sauvegarde des poids du modèle. Cela garantit un entraînement efficace et prévient le surapprentissage.

5. Algorithmes

Les réseaux de neurones convolutifs (**CNN**) sont particulièrement adaptés à la résolution du problème de prédiction de "fire" et "non fire", car ils excellent dans l'extraction automatique des caractéristiques pertinentes à partir des images, tout en capturant les motifs visuels complexes nécessaires pour différencier les deux classes.

5.1 Modifications sur Model CNN

Listing 5.1 – ancien model cnn

```
1 def build_cnn():
2     model = Sequential()
3     # 1ere convolution
4     model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)))
5     model.add(MaxPooling2D(pool_size=(2, 2)))
6     # 2eme convolution
7     model.add(Conv2D(64, (3, 3), activation='relu'))
8     model.add(MaxPooling2D(pool_size=(2, 2)))
9     # 3eme convolution
10    model.add(Conv2D(128, (3, 3), activation='relu'))
11    model.add(MaxPooling2D(pool_size=(2, 2)))
12    # Aplatissement des sorties des couches de convolution
13    model.add(Flatten())
14    model.add(Dense(256, activation='relu'))
15    # Ajouter une couche softmax avec 2 classes (Fire et Non-fire)
16    model.add(Dense(2, activation='softmax'))
17
18    return model
```

Listing 5.2 – Derniere model cnn

```
1 def cnn():
2     inp = layers.Input((224, 224, 3))
3     # 1ere convolution
4     x = Conv2D(32, (3, 3), activation='relu')(inp)
5     x = BatchNormalization()(x)
6     x = MaxPooling2D()(x)
7     # 2eme convolution
8     x = Conv2D(64, (3, 3), activation='relu')(x)
9     x = BatchNormalization()(x)
10    x = MaxPooling2D()(x)
11    # 3eme convolution
12    x = Conv2D(128, (3, 3), activation='relu')(x)
13    x = BatchNormalization()(x)
14    x = MaxPooling2D()(x)
```

```

15 # 4eme convolution
16 x = Conv2D(256, (3, 3), activation='relu')(x)
17 x = BatchNormalization()(x)
18 x = MaxPooling2D()(x)
19 # Aplatissement des sorties des couches de convolution
20 x = Flatten()(x)
21 # Couches Dense et Dropout pour la classification
22 x = Dense(256, activation='relu')(x)
23 x = Dropout(0.2)(x)
24 x = Dense(256, activation='relu')(x)
25 x = Dropout(0.2)(x)
26 # Ajouter une couche sigmoid avec 2 classes ( Fire et Non - fire )
27 x = Dense(1, activation='sigmoid')(x)
28 model_cnn = models.Model(inputs=inp, outputs=x)
29
30 return model_cnn

```

1. Ajout de la Normalisation Batch (Batch Normalization)

Modification :

Ajout de BatchNormalization() après chaque couche convolutionnelle.

Pourquoi :

La normalisation batch stabilise et accélère l'entraînement en normalisant les activations des couches intermédiaires.

Elle permet au réseau de converger plus rapidement et réduit le risque de surapprentissage.

2. Augmentation du Nombre de Filtres

Modification :

Ajout d'une quatrième couche de convolution avec 256 filtres (Conv2D(256, ...)) dans le modèle amélioré.

Pourquoi :

L'ajout d'une couche supplémentaire avec plus de filtres permet au réseau de capturer des caractéristiques plus complexes et abstraites dans les images, ce qui est crucial pour distinguer des nuances entre feu et non-feu.

3. Introduction de Dropout

Modification :

Ajout de Dropout(0.2) après les couches denses.

Pourquoi :

Dropout aide à réduire le surapprentissage (overfitting) en désactivant aléatoirement un certain pourcentage de neurones pendant l'entraînement.

Cela améliore la généralisation du modèle sur des données de validation et de test.

4. Passage de Softmax à Sigmoid pour la Sortie

Modification :

Remplacement de la dernière couche avec activation softmax (2 sorties) par une activation sigmoid (1 sortie).

Pourquoi :

Pour une classification binaire (feu/non-feu), une activation sigmoid est plus adaptée, car elle génère une probabilité pour une seule classe.

Cela simplifie également la configuration de la fonction de perte (`binary_crossentropy` au lieu de `categorical_crossentropy`).

5. Augmentation du Nombre de Paramètres dans les Couches Denses

Modification :

Ajout d'une couche dense supplémentaire avec 256 neurones.

Pourquoi :

Les couches denses apprennent des combinaisons complexes de caractéristiques extraites par les couches de convolution.

Une architecture plus profonde pour les couches denses augmente la capacité d'apprentissage, essentielle pour un problème complexe comme la détection de feu.

6. Changement de l'Architecture Entrée-Sortie

Modification :

Utilisation d'un modèle fonctionnel au lieu de `Sequential()`.

Pourquoi :

Le modèle fonctionnel permet plus de flexibilité pour des architectures avancées, comme des connexions résiduelles ou des réseaux multi-sorties.

Bien que dans ce cas, le réseau soit encore simple, le modèle fonctionnel peut être étendu à l'avenir.

7. Ajustement des Paramètres de Pooling

Modification :

Utilisation uniforme de `MaxPooling2D()` après chaque couche de convolution.

Pourquoi :

Le pooling réduit la taille des cartes de caractéristiques tout en conservant les informations essentielles, ce qui diminue la charge computationnelle et aide à éviter le surapprentissage.

Pourquoi Ces Modifications Améliorent la Performance :

Stabilisation et accélération de l'entraînement : Grâce à la normalisation batch.

Réduction de l'overfitting : Dropout et architecture optimisée réduisent le risque d'apprentissage des détails spécifiques au dataset.

Augmentation de la capacité d'apprentissage : Les couches supplémentaires permettent de capturer des caractéristiques plus complexes.

Optimisation pour une classification binaire : Une sortie sigmoïde est mieux adaptée et plus précise pour détecter une classe spécifique (feu) avec une probabilité.

5.2 Comprendre input et output Model CNN

5.2.1 1_ère couche convolution

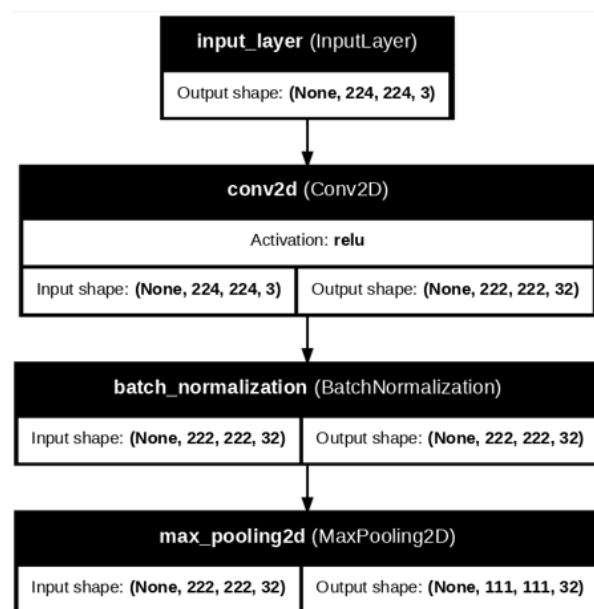


FIGURE 5.1 – 1_ère couche convolution de CNN

1. Couche d'entrée (Input Layer)

Input Shape : (None, 224, 224, 3)

- **None** représente la taille dynamique du batch.
- **224x224** est la résolution de chaque image.
- **3** correspond aux canaux de couleur (RVB).

Rôle : Accepte une image d'entrée au format 224x224x3.

2. Couche de convolution 1 (Conv2D)

Input Shape : (None, 224, 224, 3)

Output Shape : (None, 222, 222, 32)

- Réduction de 2 pixels par dimension ($224 \rightarrow 222$) à cause du noyau (**3x3**) sans padding.
- 32 filtres extraits des caractéristiques locales.

Rôle : Détecte les caractéristiques locales simples (comme les bords ou textures).

3. Normalisation Batch 1 (BatchNormalization)

Input Shape : (None, 222, 222, 32)

Output Shape : (None, 222, 222, 32)

Rôle : Normalise les activations pour stabiliser et accélérer l'apprentissage.

4. MaxPooling 1 (MaxPooling2D)

Input Shape : (None, 222, 222, 32)

Output Shape : (None, 111, 111, 32)

- Réduction de la taille spatiale par un facteur de 2 (pooling (2x2)).

Rôle : Réduit la dimension des caractéristiques, préservant les informations les plus importantes.

5.2.2 2_ème couche convolution

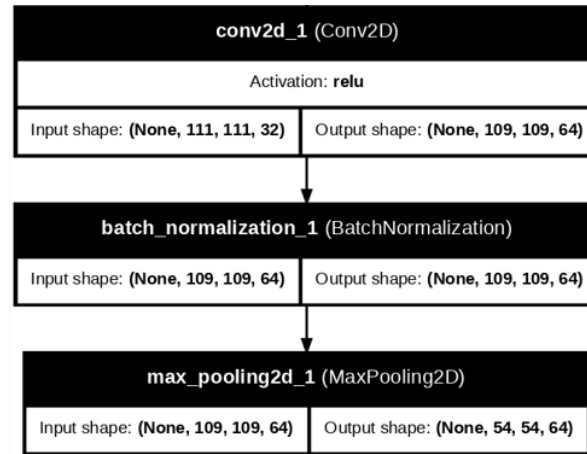


FIGURE 5.2 – 2_ème couche convolution

5. Couche de convolution 2 (Conv2D)

Input Shape : (None, 111, 111, 32)

Output Shape : (None, 109, 109, 64)

- 64 filtres avec un noyau (**3x3**) pour détecter des caractéristiques plus complexes.

Rôle : Apprend des motifs intermédiaires.

6. Normalisation Batch 2 (BatchNormalization)

Input Shape : (None, 109, 109, 64)

Output Shape : (None, 109, 109, 64)

Rôle : Stabilise les activations et améliore la convergence.

7. MaxPooling 2 (MaxPooling2D)

Input Shape : (None, 109, 109, 64)

Output Shape : (None, 54, 54, 64)

- Réduction de moitié de la taille spatiale.

Rôle : Réduit davantage la dimension des cartes de caractéristiques.

5.2.3 3_ème couche convolution

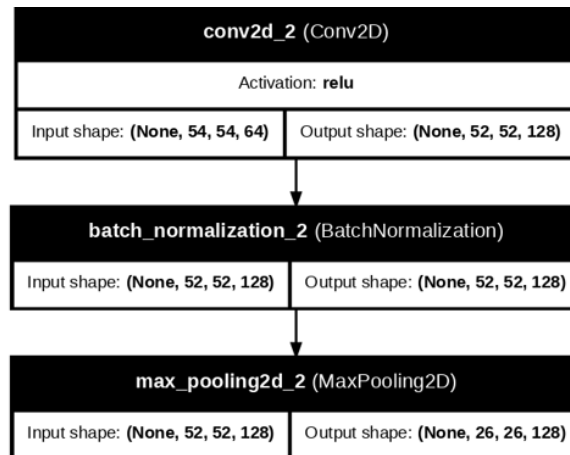


FIGURE 5.3 – 3_ème couche convolution

8. Couche de convolution 3 (Conv2D)

Input Shape : (None, 54, 54, 64)

Output Shape : (None, 52, 52, 128)

- 128 filtres avec un noyau (**3x3**).

Rôle : Capture des caractéristiques plus complexes (formes et contours).

9. Normalisation Batch 3 (BatchNormalization)

Input Shape : (None, 52, 52, 128)

Output Shape : (None, 52, 52, 128)

Rôle : Stabilise les activations.

10. MaxPooling 3 (MaxPooling2D)

Input Shape : (None, 52, 52, 128)

Output Shape : (None, 26, 26, 128)

- Réduction de moitié de la taille spatiale.

Rôle : Diminue la dimension pour réduire la complexité computationnelle.

5.2.4 4_ème couche convolution

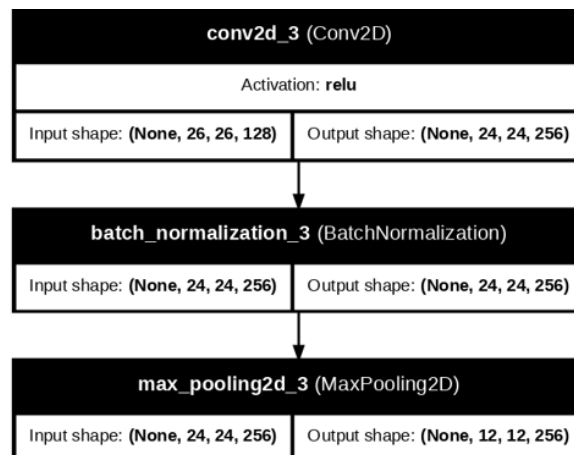


FIGURE 5.4 – 4_ème couche convolution de CNN

11. Couche de convolution 4 (Conv2D)

Input Shape : `(None, 26, 26, 128)`

Output Shape : `(None, 24, 24, 256)`

- 256 filtres pour détecter des caractéristiques de haut niveau (textures complexes).

Rôle : Apprend des combinaisons de caractéristiques abstraites.

12. Normalisation Batch 4 (BatchNormalization)

Input Shape : `(None, 24, 24, 256)`

Output Shape : `(None, 24, 24, 256)`

Rôle : Stabilise et normalise les activations.

13. MaxPooling 4 (MaxPooling2D)

Input Shape : `(None, 24, 24, 256)`

Output Shape : `(None, 12, 12, 256)`

- Réduction de moitié de la taille spatiale.

Rôle : Simplifie les données pour le passage à la partie dense.

5.2.5 Flatten et couche de classification

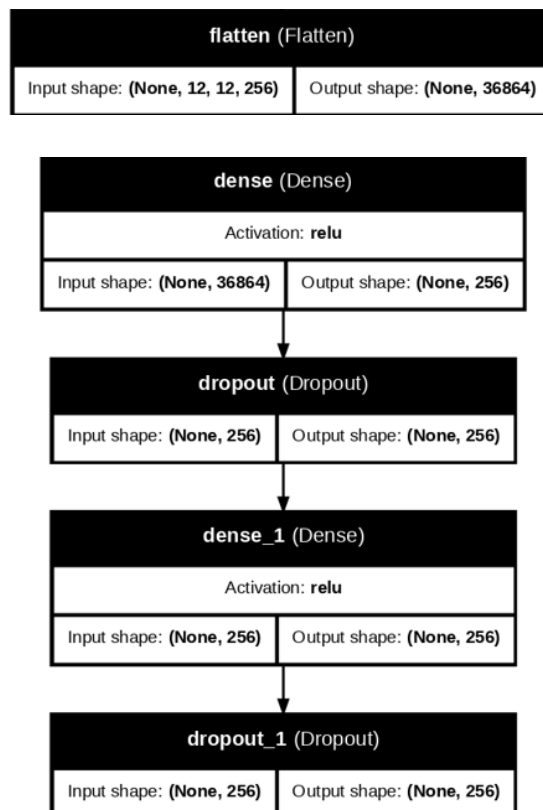


FIGURE 5.5 – Flatten et classification

14. Flatten

Input Shape : (None, 12, 12, 256)

Output Shape : (None, 36864)

- Les dimensions sont aplaties en un vecteur de **36 864** valeurs.

Rôle : Prépare les données pour les couches entièrement connectées (denses).

15. Couche dense 1 (Dense)

Input Shape : (None, 36864)

Output Shape : (None, 256)

- Apprend des combinaisons complexes de caractéristiques.

Rôle : Diminue la dimension tout en capturant des relations entre les caractéristiques.

16. Dropout 1

Input Shape : (None, 256)

Output Shape : (None, 256)

- Désactive aléatoirement 20 % des neurones.

Rôle : Réduit le surapprentissage.

17. Couche dense 2 (Dense)

Input Shape : (None, 256)

Output Shape : (None, 256)

- Une autre couche dense pour capturer davantage de relations.

Rôle : Raffine les relations apprises.

18. Dropout 2

Input Shape : (None, 256)

Output Shape : (None, 256)

- Renforce la généralisation du modèle.

Rôle : Réduit le surapprentissage en désactivant aléatoirement certains neurones.

5.2.6 Output de model

dense_2 (Dense)	
Activation: sigmoid	
Input shape: (None, 256)	Output shape: (None, 1)

FIGURE 5.6 – output

19. Couche de sortie (Dense)

Input Shape : (None, 256)

Output Shape : (None, 1)

- Une sortie unique avec une activation **sigmoid**.

Rôle : Fournit une probabilité pour la classe "**Feu**" (1) ou "**Non-Feu**" (0).

5.3 entraînement du Model

5.3.1 Les paramètres et entraînement du Model

```
1 if __name__ == "__main__":
2
3     model_cnn.compile(
4         optimizer=Adam(0.0001),
5         loss='binary_crossentropy',
6         metrics=['accuracy']
7     )
8
9     # Entraîner le modèle
10    history_data = train_model(model_cnn, train_images, val_images, epochs=10)
```

Le modèle utilise les paramètres suivants :

- **Optimiseur :** Adam(0.0001) - L'optimiseur Adam est un optimiseur adaptatif qui ajuste dynamiquement le taux d'apprentissage pour chaque paramètre. Un taux d'apprentissage de 0.0001 est spécifié pour assurer une convergence stable. Cependant, il peut entraîner une augmentation des valeurs de

`val_loss` au début de l'entraînement, car un taux d'apprentissage trop faible peut ralentir l'ajustement du modèle aux données d'entraînement. Le choix d'un taux d'apprentissage plus faible permet de minimiser les risques de sur-apprentissage, mais peut entraîner des variations plus grandes dans la fonction de perte sur les ensembles de validation.

- **Fonction de perte : `binary_crossentropy`** - La fonction de perte `binary_crossentropy` est utilisée pour la classification binaire, où l'objectif est de prédire une étiquette parmi deux classes (par exemple, feu ou non-feu). Elle est calculée en fonction de la probabilité de prédiction p et de la vérité terrain y comme suit :

$$L = -(y \cdot \log(p) + (1 - y) \cdot \log(1 - p))$$

où y est la vérité terrain (0 ou 1), et p est la probabilité prédite pour la classe "1" (feu). Cette fonction de perte pénalise fortement les prédictions éloignées de la vérité terrain, encourageant ainsi le modèle à produire des probabilités de prédiction plus précises.

- **Métrique : `accuracy`** - La métrique utilisée est la précision (`accuracy`), qui mesure la proportion de prédictions correctes parmi l'ensemble des prédictions. Elle est calculée comme suit :

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}}$$

L'entraînement est effectué sur 10 époques avec des données d'entraînement et de validation. Le taux d'apprentissage spécifié à 0.0001 permet un ajustement progressif des poids du modèle, tout en maintenant une certaine stabilité dans la convergence. Cependant, au début de l'entraînement, il est possible que la fonction de perte sur l'ensemble de validation (`val_loss`) devienne plus élevée en raison de la lente adaptation du modèle aux données d'entraînement.

Ces choix sont faits pour optimiser la performance du modèle tout en évitant un sur-apprentissage trop rapide, ce qui est crucial pour garantir une bonne généralisation aux nouvelles données.

5.3.2 Resultat d'entrainemet du Model

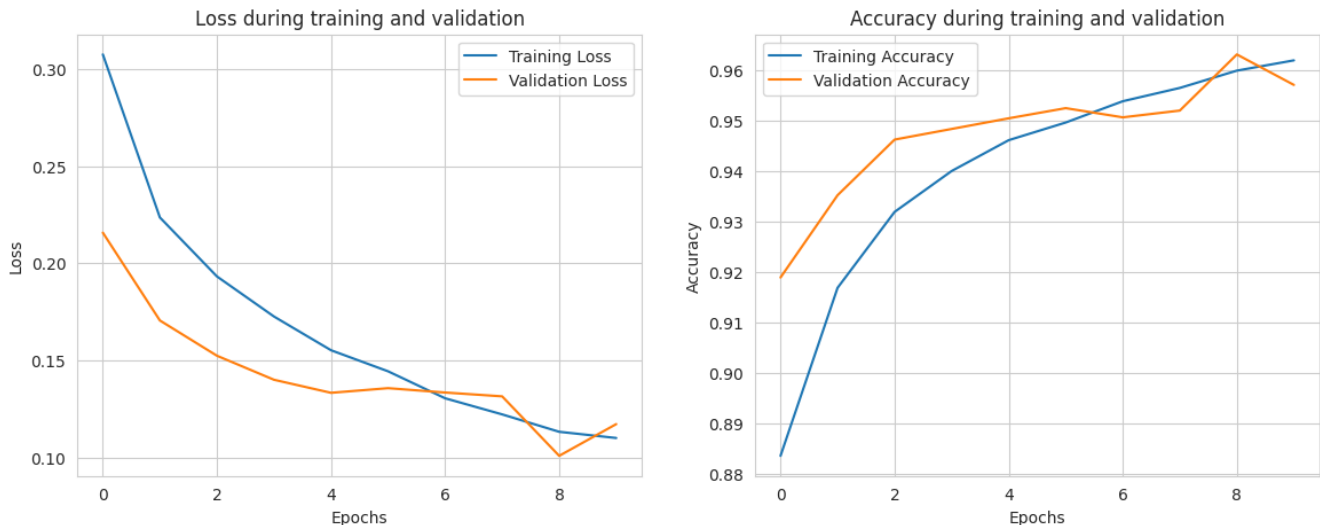


FIGURE 5.7 – Loss et Accuracy history d'entrainement

Perte (Loss) :

La perte d'entraînement diminue régulièrement, passant de 0.36 à environ 0.11, indiquant que le modèle apprend efficacement à chaque époque. La perte de validation suit une tendance similaire, atteignant des valeurs basses autour de 0.10 à l'époque 9, ce qui montre que le modèle généralise bien sur les données de validation.

Précision (Accuracy) :

La précision d'entraînement passe de 86.5 % (époque 1) à environ 96.2 % (époque 10), montrant une amélioration constante. La précision de validation atteint un maximum de 96.32 % à l'époque 9, démontrant une très bonne capacité à prédire correctement les classes "FIRE" et "NON-FIRE".

Restauration des poids :

Le modèle restaure automatiquement les meilleurs poids basés sur les performances de validation, garantissant une optimisation des résultats et minimisant le risque de surapprentissage.

Ces résultats indiquent que le modèle est performant et bien ajusté pour votre dataset de détection de "FIRE" et "NON-FIRE".

5.4 Test du Modèle

5.4.1 Prediction

```
1 test_predictions = model_cnn.predict(test_images)
2 test_labels = test_df['Label'].values
3
4 binary_predictions = np.round(test_predictions).flatten()
5
6 print("Classification Report:")
7 print(classification_report(test_labels, binary_predictions))
8
9 accuracy_value = accuracy(test_labels, binary_predictions)
10 precision_value = precision(test_labels, binary_predictions)
11 recall_value = recall(test_labels, binary_predictions)
12 f1_score_value = f1_score(test_labels, binary_predictions)
13
14 print("\nMetrics:")
15 print(f"Accuracy: {accuracy_value:.2f}%")
16 print(f"Precision: {precision_value:.2f}%")
17 print(f"Recall: {recall_value:.2f}%")
18 print(f"F1-Score: {f1_score_value:.2f}%")
```

1. Prédictions du modèle sur les données de test :

- La ligne `test_predictions = model_cnn.predict(test_images)` effectue des prédictions sur les images de test en utilisant le modèle entraîné `model_cnn`. Cela renvoie un tableau de valeurs de probabilité pour chaque image, indiquant la probabilité d'appartenir à la classe "FIRE" ou "NON-FIRE".

2. Récupération des étiquettes réelles des images de test :

- `test_labels = test_df['Label'].values` extrait les véritables étiquettes (ou classes) des images de test à partir du DataFrame `test_df`. Ces étiquettes sont utilisées pour comparer avec les prédictions générées par le modèle.

3. Conversion des prédictions en classes binaires :

- La ligne `binary_predictions = np.round(test_predictions).flatten()` arrondit les valeurs de probabilité des prédictions à la classe la plus proche (0 ou 1) et aplati les résultats en un tableau unidimensionnel. Les valeurs de probabilité proches de 1 sont arrondies à 1 (classe "FIRE"), tandis que celles proches de 0 sont arrondies à 0 (classe "NON-FIRE").

4. Affichage du rapport de classification :

- La ligne `print("Classification Report:")` affiche le texte indiquant que le rapport de classification va suivre.
- `print(classification_report(test_labels, binary_predictions))` génère et affiche un rapport détaillant la précision, le rappel et le F1-score pour chaque classe (ici, "FIRE" et "NON-FIRE"), basé sur les comparaisons entre les étiquettes réelles et les prédictions binaires.

5. Calcul des métriques d'évaluation :

- `accuracy_value = accuracy(test_labels, binary_predictions)` calcule l'*accuracy* (précision globale) en utilisant la fonction `accuracy`, qui mesure la proportion de prédictions correctes par rapport à l'ensemble des prédictions.
- `precision_value = precision(test_labels, binary_predictions)` calcule la *précision*, c'est-à-dire la proportion de prédictions positives correctes parmi toutes les prédictions positives.
- `recall_value = recall(test_labels, binary_predictions)` calcule le *rappel*, c'est-à-dire la proportion d'instances positives correctement identifiées parmi toutes les instances positives réelles.
- `f1_score_value = f1_score(test_labels, binary_predictions)` calcule le *F1-score*, une mesure combinée qui équilibre la précision et le rappel. Il est particulièrement utile lorsqu'il y a un déséquilibre entre les classes.

3.2. Résultat :

```
288/288 ————— 120s 414ms/step
Classification Report:
              precision    recall  f1-score   support

     0       0.96      0.99      0.97       6523
     1       0.98      0.89      0.93       2664

 accuracy          0.96          0.96          0.96       9187
 macro avg         0.97          0.94          0.95       9187
 weighted avg      0.96          0.96          0.96       9187
```

```
Metrics:
Accuracy: 96.22%
Precision: 0.98
Recall: 0.89
F1-Score: 0.93
```

FIGURE 5.8 – Resultat De Test

1. Analyse des performances globales :

- **Accuracy (Exactitude) :** Le modèle atteint une exactitude globale de 96.22 %, ce qui signifie qu'il a correctement classé environ 96 % des images dans le dataset de test. Cela confirme que le modèle est bien généralisé pour les nouvelles données.

2. Analyse des classes :

— Classe 0 (NON-FIRE) :

- **Précision :** 0.96 (96 %) Cela signifie que sur toutes les prédictions faites pour la classe "NON-FIRE", 96 % sont correctes. Le modèle évite bien les fausses détections de feu dans des images sans feu.
- **Rappel :** 0.99 (99 %) Parmi toutes les images réellement "NON-FIRE", le modèle en a identifié 99 %. Le modèle est donc très efficace pour capturer presque toutes les images sans feu.
- **F1-Score :** 0.97 (97 %) Une combinaison équilibrée de précision et de rappel pour cette classe.

— Classe 1 (FIRE) :

- **Précision :** 0.98 (98 %) Cela montre que le modèle est excellent pour détecter les images contenant un feu, avec peu de fausses alertes.
- **Rappel :** 0.89 (89 %) Le modèle identifie 89 % des images contenant un feu, ce qui est bon, mais cela indique que certaines images de feu ne sont pas détectées (faux négatifs).
- **F1-Score :** 0.93 (93 %) Une bonne performance globale, mais légèrement inférieure à la classe "NON-FIRE", probablement en raison du rappel plus bas.

3. Analyse globale (Macro & Weighted) :

- **Macro Average :** Avec une précision moyenne de 0.97, un rappel moyen de 0.94 et un F1-score moyen de 0.95, le modèle montre un bon équilibre dans le traitement des deux classes, indépendamment de leur taille.
- **Weighted Average :** La moyenne pondérée (0.96 pour toutes les métriques) reflète que la performance globale du modèle est solide, tout en tenant compte des proportions des classes dans le dataset.

5.5 évaluation du Modèle

5.5.1 Préparation de l'Image

```
1 model = load_model(r'C:\Users\Lenovo\Desktop\version_final\cnn\Cnn.keras')
2
3 def prepare_image(file):
4     img_path = ''
5     img = image.load_img(img_path + file, target_size=(224, 224))
6     img_array = image.img_to_array(img)
7     img_array = img_array / 255.0
8     img_array_expanded_dims = np.expand_dims(img_array, axis=0)
9     return img_array_expanded_dims
```

1. Chargement du modèle :

La première ligne charge un modèle préalablement sauvegardé depuis un fichier spécifié, ici le fichier **Cnn.keras** qui contient l'architecture et les poids du modèle entraîné.

2. Fonction `prepare_image(file)` :

Cette fonction est utilisée pour préparer une image avant de la passer à travers le modèle pour effectuer des prédictions. Elle effectue les étapes suivantes :

- Chargement de l'image :

`image.load_img(img_path + file, target_size=(224, 224))` charge l'image à partir du chemin spécifié (en concaténant `img_path` et le nom du fichier `file`) et la redimensionne à une taille de 224×224 pixels, ce qui est généralement la taille attendue par de nombreux modèles CNN.

- Conversion de l'image en tableau :

`image.img_to_array(img)` convertit l'image chargée en un tableau numpy de dimensions (hauteur, largeur, canaux de couleur), ce qui permet de traiter les valeurs des pixels sous forme de données numériques.

- Normalisation de l'image :

`img_array = img_array / 255.0` normalise les valeurs des pixels de l'image en les divisant par 255 (pour que les valeurs soient dans la plage $[0, 1]$). Cela est souvent effectué pour améliorer la convergence du modèle pendant l'entraînement.

- Ajout d'une dimension supplémentaire :

`np.expand_dims(img_array, axis=0)` ajoute une dimension supplémentaire à la matrice, transformant ainsi le tableau de forme (224, 224, 3) en (1, 224, 224, 3). Cette dimension supplémentaire correspond au lot d'images (batch), même si nous n'avons qu'une seule image dans ce cas, le modèle s'attend à des entrées sous forme de lot.

- Retour de l'image préparée :

Enfin, la fonction retourne l'image préparée, prête à être donnée en entrée du modèle pour la prédiction.

En résumé, cette fonction permet de préparer les images d'entrée pour qu'elles puissent être utilisées par le modèle de manière compatible avec la structure attendue par ce dernier.

5.5.2 Prediction

```
1 preprocessed_image = prepare_image(r"C:\Users\Lenovo\Desktop\version_final\evaluation\1.  
   jpg")  
2 predictions = model.predict(preprocessed_image)  
3 prediction_value = predictions[0][0]  
4  
5 if prediction_value > 0.5:  
6     fire_percentage = prediction_value * 100  
7     nonfire_percentage = (1 - prediction_value) * 100  
8 else:  
9     fire_percentage = prediction_value * 100  
10    nonfire_percentage = (1 - prediction_value) * 100  
11  
12 print("Fire_:", round(fire_percentage, 2), "%", "|", "Non_Fire_:", round(  
    nonfire_percentage, 2), "%")  
13  
14 if prediction_value > 0.5:  
15     print("Fire_detected!")
```



```

16 else:
17     print("No fire detected.")

```

Le code présenté effectue une prédiction pour déterminer la probabilité qu'une image représente un incendie ou non. Voici les étapes détaillées :

- **Chargement et prétraitement de l'image** : La fonction `prepare_image` est utilisée pour charger et prétraiter une image à partir du chemin spécifié (`1.jpg`). Cette fonction ajuste l'image afin qu'elle soit compatible avec le modèle de prédiction.
- **Prédiction avec le modèle** : La fonction `model.predict(preprocessed_image)` exécute une prédiction sur l'image prétraitée. La sortie est stockée dans la variable `predictions`, et la probabilité principale est extraite avec `predictions[0][0]`.
- **Calcul des pourcentages** :
 - Si la valeur de prédiction (`prediction_value`) est supérieure à 0.5, cela signifie que le modèle estime qu'il est plus probable qu'il y ait un incendie. Les pourcentages sont calculés comme suit :
 - Pourcentage de feu : `prediction_value * 100`
 - Pourcentage de non-feu : `(1 - prediction_value) * 100`
 - Si la valeur de prédiction est inférieure ou égale à 0.5, cela indique qu'il est plus probable qu'il n'y ait pas d'incendie. Les pourcentages sont alors calculés ainsi :
 - Pourcentage de feu : `prediction_value * 100`
 - Pourcentage de non-feu : `(1 - prediction_value) * 100`
- **Affichage des résultats** : Les pourcentages calculés sont arrondis et affichés dans la console sous le format : `"Fire : X % | Non Fire : Y %"`.
- **Détection finale** : Une détection finale est effectuée avec le seuil de 0.5 :
 - Si `prediction_value > 0.5`, le message `"Fire detected!"` est affiché.
 - Sinon, le message `"No fire detected."` est affiché.

5.5.3 Exemples

Exemple Fire



FIGURE 5.9 – Fire Detiction

Après l'exécution du code de prédiction, les résultats obtenus sont les suivants :

- **Probabilité de Feu : 70.32 %** Le modèle estime qu'il y a une forte probabilité que l'image représente un incendie. Cette valeur dépasse le seuil de 50 %, ce qui indique une détection positive de feu.

- **Probabilité de Non-Feu : 2.97 %** Cette valeur a été calculée à l'aide de la formule :

$$\text{NonFire_Percentage} = (1 - \text{prediction_value}) \times 100$$

Cependant, cette valeur semble incohérente avec la logique classique des probabilités où la somme des pourcentages de *Feu* et de *Non-Feu* devrait être égale à 100 %.

- **Détection Finale :** Étant donné que la probabilité de feu (70.32 %) est supérieure au seuil de 50 %, le modèle conclut que l'image montre un incendie. Le message "**Fire detected!**" a donc été affiché.

Le calcul de la probabilité de *Non-Feu* (2.97%) est affecté par l'utilisation d'un facteur de pondération ($\times 100$) dans le code. Cela ne suit pas la logique classique où les probabilités complémentaires devraient totaliser 100 %. Si cette incohérence n'est pas intentionnelle, il serait pertinent de modifier la formule pour garantir une cohérence mathématique.

Le modèle indique avec une forte probabilité (70.32%) que l'image représente un incendie, ce qui a déclenché une détection positive.

Exemple Non-Fire

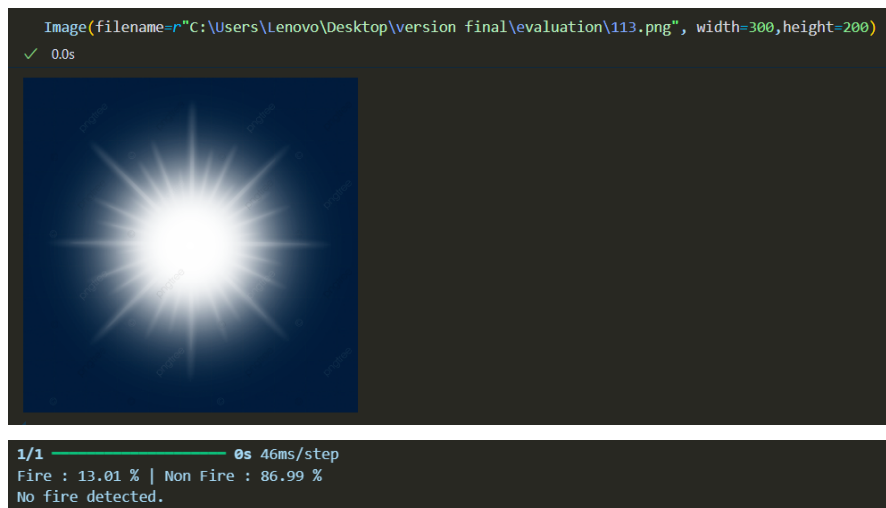


FIGURE 5.10 – Non Fire Detiction

Après l'exécution du code de prédiction, les résultats obtenus sont les suivants :

- **Probabilité de Feu : 13.01 %** Le modèle estime qu'il y a une faible probabilité que l'image représente un incendie. Cette valeur est bien inférieure au seuil de 50 %, ce qui indique qu'il est peu probable que l'image montre un feu.
- **Probabilité de Non-Feu : 86.99 %** Cette valeur a été calculée comme complément logique de la probabilité de *Feu* :

$$\text{NonFire_Percentage} = (1 - \text{prediction_value}) \times 100$$

Ici, les calculs sont cohérents avec les probabilités standards où la somme des pourcentages de *Feu* et de *Non-Feu* est égale à 100 %.

- **Détection Finale :** Étant donné que la probabilité de feu (13.01 %) est inférieure au seuil de 50 %, le modèle conclut qu'il n'y a pas d'incendie détecté. Le message "**No fire detected.**" a donc été affiché.

Le modèle indique avec une forte probabilité (86.99%) que l'image ne représente pas un incendie, ce qui justifie la décision de ne pas signaler de feu.

6. Comparaison

Les modèles de comparaison

AlexNet est particulièrement adapté à la résolution du problème de prédiction entre "feu" et "non feu" grâce à sa capacité à extraire automatiquement des caractéristiques pertinentes à partir des images. Conçu avec des couches convolutionnelles profondes et des fonctions d'activation non-linéaires, AlexNet excelle dans l'identification des motifs visuels complexes, tels que les contours irréguliers, les couleurs distinctives (comme le rouge et l'orange), et les textures spécifiques au feu.

6.1 Description des blocs de l'architecture AlexNet

Bloc 1 (Convolution, Normalisation, Pooling)

- **Convolution 2D** :
 - Applique 96 filtres de taille 11×11 .
 - Stride = 4, ce qui réduit la taille spatiale de l'image.
 - Activation **ReLU** introduit la non-linéarité.
- **Batch Normalization** : Normalise les activations pour accélérer l'entraînement et améliorer la stabilité.
- **MaxPooling** : Réduit davantage la taille spatiale avec une fenêtre de 3×3 et un stride de 2.

Bloc 2 (Convolution, Normalisation, Pooling)

- **Convolution 2D** :
 - Applique 256 filtres de taille 5×5 .
 - Padding = **same** : Maintient la taille spatiale après la convolution.
 - Activation **ReLU**.
- **Batch Normalization** : Améliore la convergence.
- **MaxPooling** : Réduit encore la taille spatiale.

Bloc 3 (Convolutions Profondes et Pooling)

- **Convolutions 2D** :
 - Trois convolutions successives avec des tailles de filtre 3×3 .
 - Les 384, 384, et 256 filtres permettent de capturer des caractéristiques de haut niveau, comme des motifs complexes ou des relations entre objets.
 - Activation **ReLU**.
- **MaxPooling** : Réduit la taille spatiale pour produire un vecteur caractéristique compact.

Fully Connected Layers

- **Flatten** : Aplatit la sortie des couches précédentes en un vecteur 1D pour l'entrée dans des couches denses.
- **Dense Layers** :
 - Deux couches entièrement connectées avec 4096 neurones chacune.
 - Activation **ReLU** pour la non-linéarité.
- **Dropout** : Désactive aléatoirement 50% des neurones pendant l'entraînement pour réduire le surapprentissage.

Listing 6.1 – Définition du modèle AlexNet en Python

```
1 def AlexNet():
2     inp = layers.Input((224, 224, 3))
3
4     # Bloc 1
5     x = layers.Conv2D(96, kernel_size=11, strides=4, activation='relu')(inp)
6     x = layers.BatchNormalization()(x)
7     x = layers.MaxPooling2D(pool_size=3, strides=2)(x)
8
9     # Bloc 2
10    x = layers.Conv2D(256, kernel_size=5, padding='same', activation='relu')(x)
11    x = layers.BatchNormalization()(x)
12    x = layers.MaxPooling2D(pool_size=3, strides=2)(x)
13
14    # Bloc 3
15    x = layers.Conv2D(384, kernel_size=3, padding='same', activation='relu')(x)
16    x = layers.Conv2D(384, kernel_size=3, padding='same', activation='relu')(x)
17    x = layers.Conv2D(256, kernel_size=3, padding='same', activation='relu')(x)
18    x = layers.MaxPooling2D(pool_size=3, strides=2)(x)
19
20    # Fully Connected Layers
21    x = layers.Flatten()(x)
22    x = layers.Dense(4096, activation='relu')(x)
23    x = layers.Dropout(0.5)(x)
24    x = layers.Dense(4096, activation='relu')(x)
25    x = layers.Dropout(0.5)(x)
26
27    # Couche de sortie pour la classification binaire
28    x = layers.Dense(1, activation='sigmoid')(x)
29
30    model = models.Model(inputs=inp, outputs=x)
31    return model
32
33 model_Alex = AlexNet()
34 model_Alex.summary()
```

Résultat :

Classification Report:					
	precision	recall	f1-score	support	
0	0.97	0.98	0.98	6523	
1	0.96	0.92	0.94	2664	
accuracy			0.96	9187	
macro avg	0.96	0.95	0.96	9187	
weighted avg	0.96	0.96	0.96	9187	

```
Confusion Matrix:
[[6410  113]
 [ 209 2455]]
```

```
Metrics:
Accuracy: 96.50%
Precision: 0.96
Recall: 0.92
F1-Score: 0.94
```

FIGURE 6.1 – Résultats des tests

VGGNet est particulièrement adapté à la résolution du problème de prédiction entre "feu" et "non feu" grâce à sa capacité à extraire automatiquement des caractéristiques pertinentes à partir des images. Conçu avec des couches convolutionnelles profondes et des fonctions d'activation non-linéaires, VGGNet excelle dans l'identification des motifs visuels complexes, tels que les contours irréguliers, les couleurs distinctives (comme le rouge et l'orange), et les textures spécifiques au feu.

6.2 Description des blocs de l'architecture VGGNet

Bloc 1 (Convolution, Normalisation, Pooling)

- **Convolution 2D** :
 - Applique 64 filtres de taille 3×3 .
 - Padding = **same** : Maintient la taille spatiale après la convolution.
 - Activation **ReLU**.
- **Convolution 2D** :
 - Applique 64 filtres supplémentaires de taille 3×3 .
 - Padding = **same**.
 - Activation **ReLU**.
- **Batch Normalization** : Normalise les activations pour accélérer l'entraînement et améliorer la stabilité.
- **MaxPooling** : Réduit la taille spatiale avec une fenêtre de 2×2 et un stride de 2.

Bloc 2 (Convolution, Normalisation, Pooling)

- **Convolution 2D** :
 - Applique 128 filtres de taille 3×3 .
 - Padding = **same**.

- Activation **ReLU**.
- **Convolution 2D** :
 - Applique 128 filtres supplémentaires de taille 3×3 .
 - Padding = **same**.
 - Activation **ReLU**.
- **Batch Normalization** : Améliore la convergence.
- **MaxPooling** : Réduit la taille spatiale.

Bloc 3 (Convolutions Profondes et Pooling)

- **Convolutions 2D** :
 - Trois convolutions successives avec des filtres de taille 3×3 .
 - Les 256 filtres permettent de capturer des caractéristiques de plus haut niveau.
 - Activation **ReLU**.
- **Batch Normalization** : Normalise les activations.
- **MaxPooling** : Réduit la taille spatiale pour créer un vecteur caractéristique compact.

Bloc 4 (Convolutions Profondes et Pooling)

- **Convolutions 2D** :
 - Trois convolutions successives avec des filtres de taille 3×3 et 512 filtres.
 - Activation **ReLU**.
- **Batch Normalization** : Améliore la stabilité et la convergence.
- **MaxPooling** : Réduit encore la taille spatiale.

Bloc 5 (Convolutions Profondes et Pooling)

- **Convolutions 2D** :
 - Trois convolutions successives avec des filtres de taille 3×3 et 512 filtres.
 - Activation **ReLU**.
- **Batch Normalization** : Normalise les activations.
- **MaxPooling** : Réduit la taille spatiale pour créer un vecteur caractéristique compact.

Fully Connected Layers

- **Flatten** : Aplatit la sortie des couches précédentes en un vecteur 1D pour l'entrée dans des couches denses.
- **Dense Layers** :
 - Deux couches entièrement connectées avec 4096 neurones chacune.
 - Activation **ReLU** pour la non-linéarité.
- **Dropout** : Désactive aléatoirement 50% des neurones pendant l'entraînement pour réduire le surapprentissage.

Couche de Sortie

- **Dense Layer** : La couche finale avec un seul neurone et activation **sigmoid** pour la classification binaire.

```

1 def VGGNet():
2     inp = layers.Input((224, 224, 3))
3
4     # Bloc 1
5     x = layers.Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same')(inp)
6     x = layers.Conv2D(64, kernel_size=(3, 3), activation='relu', padding='same')(x)
7     x = layers.BatchNormalization()(x)
8     x = layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)
9
10    # Bloc 2
11    x = layers.Conv2D(128, kernel_size=(3, 3), activation='relu', padding='same')(x)
12    x = layers.Conv2D(128, kernel_size=(3, 3), activation='relu', padding='same')(x)
13    x = layers.BatchNormalization()(x)
14    x = layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)
15
16    # Bloc 3
17    x = layers.Conv2D(256, kernel_size=(3, 3), activation='relu', padding='same')(x)
18    x = layers.Conv2D(256, kernel_size=(3, 3), activation='relu', padding='same')(x)
19    x = layers.Conv2D(256, kernel_size=(3, 3), activation='relu', padding='same')(x)
20    x = layers.BatchNormalization()(x)
21    x = layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)
22
23    # Bloc 4
24    x = layers.Conv2D(512, kernel_size=(3, 3), activation='relu', padding='same')(x)
25    x = layers.Conv2D(512, kernel_size=(3, 3), activation='relu', padding='same')(x)
26    x = layers.Conv2D(512, kernel_size=(3, 3), activation='relu', padding='same')(x)
27    x = layers.BatchNormalization()(x)
28    x = layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)
29
30    # Bloc 5
31    x = layers.Conv2D(512, kernel_size=(3, 3), activation='relu', padding='same')(x)
32    x = layers.Conv2D(512, kernel_size=(3, 3), activation='relu', padding='same')(x)
33    x = layers.Conv2D(512, kernel_size=(3, 3), activation='relu', padding='same')(x)
34    x = layers.BatchNormalization()(x)
35    x = layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2))(x)
36
37    # Fully connected layers
38    x = layers.Flatten()(x)
39    x = layers.Dense(4096, activation='relu')(x)
40    x = layers.Dropout(0.5)(x)
41    x = layers.Dense(4096, activation='relu')(x)
42    x = layers.Dropout(0.5)(x)
43    x = layers.Dense(1, activation='sigmoid')(x)
44
45    model_VGG = models.Model(inputs=inp, outputs=x)
46
47    return model_VGG
48
49 model_VGG = VGGNet()
50 model_VGG.summary()

```

288/288 115s 391ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.99	0.97	6523
1	0.97	0.89	0.93	2664
accuracy			0.96	9187
macro avg	0.96	0.94	0.95	9187
weighted avg	0.96	0.96	0.96	9187

Confusion Matrix:
[[6446 77]
[280 2384]]

Metrics:
Accuracy: 96.11%
Precision: 0.97
Recall: 0.89
F1-Score: 0.93

FIGURE 6.2 – Résultats des tests

6.3 Tableau de Comparaison

Précision des Modèles	
Modèles	Précision (%)
ALEXNET	96.50
VGG	96.11
CNN	96.22

TABLE 6.1 – Précision des modèles CNN

7. Conclusion

Le projet de détection de feu réalisé dans le cadre de ce travail a permis de développer un système automatisé capable d'identifier la présence de feu dans des images et vidéos à l'aide de réseaux neuronaux convolutionnels (CNN). En combinant des techniques avancées de traitement d'images et d'apprentissage profond, le modèle a démontré une capacité à détecter avec précision des incendies, même dans des conditions variées (jour, nuit, divers types d'éclairage). L'intégration d'un système de notification via SMTP assure une réponse rapide en cas de détection, offrant ainsi une aide précieuse dans la prévention et la gestion des incendies.

Les améliorations envisagées, telles que l'ajout de la géolocalisation et l'amélioration de la précision grâce à l'augmentation des données, permettront de renforcer encore l'efficacité du modèle. En outre, l'intégration d'une interface utilisateur avec Flask pour l'interaction en temps réel rendra l'application accessible et conviviale. Ce projet ouvre la voie à des applications plus larges dans la détection des incendies, avec un impact potentiel dans divers secteurs, y compris la sécurité publique, l'industrie et l'agriculture.